

A whole-house Home Assistant control system

Three screens, one design language, and the twenty-odd things that went wrong

A desk kiosk, a wall tablet and a phone dashboard driving lighting, blinds, heating, multi-room audio, a full AV stack, solar and battery storage, an EV charger, cameras, a door lock and a UPS.

CODE AND DOCUMENTATION AIDED BY CLAUDE AND COWORK

The dashboards, scripts, templates and this document were built with substantial help from **Claude** and **Cowork** (Anthropic). Code was generated, debugged and refined interactively; everything here was then tested in a real house on real hardware. Read it as a worked example rather than as hand-written reference code, and test anything you copy before trusting it.

Build notes and working code

Published for the Home Assistant community

Contents

1. Why this exists

What it covers, and who it is for

2. The system at a glance

Hardware, three surfaces, one philosophy

3. A design language

Two colours, three sizes, and why that is enough

4. Surface one: the desk kiosk

Widescreen layout, auto-rotation, a countdown ring

5. Surface two: the wall tablet

Panel views, tile templates, activity logic

6. Surface three: the phone

Smaller tiles, subviews, an audio zone picker

7. The energy flow card

And the trap of doing arithmetic on sensors

8. Scripts that feel instant

Parallel branches, and the discipline of turning off

9. Watchdogs

Noticing when things quietly stop

10. Styling inside shadow DOM

The two findings that cost me an evening each

11. Moving to new hardware

A migration, and the four things that broke

12. Everything I would tell myself

The collected lessons, in one place

13. The code

What is in the accompanying folder

A. The kiosk host application

A Windows appliance for the panel - full source included

CODE AND DOCUMENTATION AIDED BY CLAUDE AND COWORK

Stated plainly, because it matters for how you read this: the dashboards, scripts, card templates and this booklet were produced with substantial help from **Claude** and **Cowork** (Anthropic), working interactively over several months. I described what I wanted, the AI wrote and debugged the code, and I tested every bit of it in the house.

What that means in practice: the **patterns and techniques are sound** because they run here daily, and every fault described was one I actually hit. But this is not hand-crafted reference code reviewed by a committee.

Test anything you copy.

The architecture, hardware choices, design language and every judgement about what was worth building are mine.

Why this exists

There is no shortage of Home Assistant tutorials. What there is less of is a worked example of a **complete** system: several screens that share a visual language, a set of scripts that behave predictably, and an honest account of the things that broke along the way.

This booklet is that account. It documents three dashboards and the code behind them, but the parts I would most like you to read are the failures. Nearly every hour I lost was spent on a system that looked healthy while being subtly wrong: a lock reporting "locked" when it was open, a card telling me my house used no electricity, a doorbell that had silently gone deaf.

Those failures share a shape, and once you can recognise it you will save yourself a great deal of time.

What you will find here

- **Patterns you can lift directly** - tile templates, activity scripts, a page-rotation controller, watchdog automations.
- **Techniques that are hard to discover** - how to recolour Home Assistant's own sliders, how to animate a progress ring inside a card, why one obvious approach to both of those fails silently.
- **Design reasoning** - why two colours beat eight, why a wall panel needs different rules from a phone.
- **Debugging stories** - what stale data looks like, and how to tell "broken" from "quiet".

What you will not find

Not a copy of my configuration. My entity IDs, source strings and sensor names are specific to my hardware, and pasting them into your system would achieve nothing. Everything in the code folder is genericised, commented, and written to be adapted.

Nor is it a beginner guide. It assumes you have Home Assistant running, know your way around YAML, and have installed something from HACS before.

THE ONE PREREQUISITE

Almost everything visual here uses **custom:button-card** from HACS. It is the single most useful frontend addition I have installed: any tile, any layout, any logic, driven by JavaScript templates. If you have not used it, start there. **card-mod** is the other one worth having, for the times you need to style a native card rather than replace it.

The system at a glance

The house has three control surfaces, each with a different job. That distinction matters more than any individual card, because it determines what belongs where.

Desk kiosk	A widescreen panel on a mini PC, always on, viewed from a metre away while doing something else. Shows status : energy, work calendar, system health, news. Cycles pages on a timer. Rarely touched.
Wall tablet	A tablet on a stand in the lounge. Shows the room I am in : lights, blinds, heating, the AV stack, music. Big tiles, tapped with a thumb while walking past.
Phone	Goes everywhere, so it can assume nothing about location . Same controls, smaller tiles, no default room, and detail pages hidden behind subviews so the top level stays short.

The kit, with tags

Named explicitly so you can tell at a glance whether any of this applies to you. If your brands differ, the patterns still hold - only the entity IDs change.

Energy

GivEnergy inverter + battery	Read and controlled over local Modbus via the GivTCP add-on, which publishes to MQTT. No cloud dependency - which turned out to matter when the manufacturer entered administration.
GivEnergy EV charger	Also via GivTCP. Its local API stalls roughly daily; see chapter nine.
Octopus Energy	HACS integration. Time-of-use tariff with a cheap overnight window, which the battery charging schedule is built around.
Axle Energy	HACS integration for grid flexibility events - the battery exports on demand during a paid event.

Audio and video

LG OLED television	Native webostv integration, plus wake-on-LAN to power it on. HDMI-CEC deliberately disabled.
Onkyo TX-NR6100 amplifier	Native onkyo integration over eISCP. Its source string is what drives every activity tile.
Sonos Port and Sonos bar	Native integration. The Port is wired into the amplifier, so lounge music comes through the AV system; the bar is a separate zone.
Amazon Fire TV Stick	Android TV integration over ADB. Slow and unreliable enough that it is being replaced with an Apple TV.
Xbox Series S	Official xbox integration using Home Assistant Cloud account linking - no Azure app registration needed, contrary to a lot of older advice.
Panasonic Blu-ray player	No network control at all, so a SwitchBot IR blaster provides power. One-way, so state is always unknown.

Lighting, blinds and plugs

Tuya lights and plugs	Tuya cloud integration. Needs re-authenticating after a server migration.
Govee lamps	Govee integration for the two lounge floor lamps.
Somfy RTS blinds	Via an Overkiz gateway. One-way radio , so position is never reported - use explicit open and close, never toggle.
TP-Link Tapo	Plugs, plus an H100 hub with tap buttons. The hub uses sub-GHz radio, so range through walls is generous.

Climate, cleaning and grounds

Google Nest thermostat	Native integration. Remember <code>hvac_action</code> , not state.
BlueAir 411i Max purifier	Native integration - fan, status LED, air quality and filter life.
Roborock Qrevo Edge	Native integration, with per-room cleaning scripts for normal and deep passes.
Segway Navimow	Robot mower, via HACS.

Security

Ring doorbell and cameras	Native integration. The push listener dies silently - hence the watchdog in chapter nine. Being replaced by a wired system.
Nuki smart lock	Over MQTT straight to my own broker, which is far better than a cloud callback. See chapter eleven for how a restore broke it.
Reolink NVR and cameras	Wired PoE, planned replacement for the cloud cameras. The integration supports adding the NVR directly and enumerates each channel.

Infrastructure

Home Assistant OS	On a small x86 mini PC, wired. Previously a thin client - chapter eleven covers the move.
Eaton 3S 700 UPS	Monitored over USB using the NUT add-on plus the Network UPS Tools integration. No network card at this size; the RJ45 sockets on the back are surge pass-through, not management.
Mosquitto broker	Add-on. Note its logins may not survive a backup restore.
HASS.Agent	On the Windows desktop, giving shutdown and restart buttons on the dashboard.
A custom bridge	A small process on the desktop posting work data - calendar, mail counts, ticket counts, news - into Home Assistant as sensors over the REST API.
Corsair XENEON Edge	The widescreen kiosk panel, driven by a purpose-built WPF host application.

Frontend add-ons (all HACS)

button-card	Does nearly all the visual work here. Essential.
card-mod	For styling Home Assistant's own cards - see chapter ten.
kiosk-mode	Hides the header on the tablet dashboard.
custom-brand-icons	Device logos - Fire TV, Sonos, Xbox, Onkyo and so on, with the phu: prefix.

One principle above the others

Prefer local control. The battery is read over Modbus on my own network, not through the manufacturer's API. That decision looked like mild paranoia when I made it. Then the manufacturer went into administration, and it became the reason my storage system still works.

The same reasoning applies at a smaller scale throughout. A doorbell whose events arrive by cloud push has a listener that can die silently. A lock that publishes to your own MQTT broker does not. Where you have the choice, take the local one - it is usually faster too.

AND A COROLLARY

Do not update firmware on hardware you cannot replace. My inverter manufacturer no longer exists, so a failed flash would mean buying a new inverter and paying an installer. The upside of an update is a minor bug fix; the downside is a very expensive paperweight. If there is an auto-update setting on kit like that, turn it off.

A design language

The single change that most improved these dashboards was throwing away colour.

An earlier version used amber for media, brown for heating, purple for cleaning, teal for blinds. Each choice was defensible and the result was a fruit machine. Worse, it was *slower to read*: with eight colours in play, none of them means anything.

Two colours, one job each

Linen #ddd6c8	Off, closed, idle, at rest. The default state of everything.
Olive #94a860	On, open, active, running. Something is happening.
Red #ff5656	Reserved. Warnings, and the "everything off" button. Used sparingly enough that it still registers.

Labels use slightly muted variants (#cfc7b6 and #a9bd76) so the icon reads first and the text second.

Backgrounds are a barely visible white wash - `rgba(255, 255, 255, 0.05)` - on a dark theme, which gives tile edges without drawing borders.

From across a room you can now answer "is anything on?" without reading a single word.

Three sizes, chosen by distance

Wall tablet	150px tiles, 54px icons, 16px labels. Sized for a thumb and for reading at two metres.
Phone	126px tiles, 44px icons, 14px labels. Slightly smaller, still comfortably tappable.
Secondary	104px tiles, 34px icons, 13px labels. Favourites and zone pickers - things you look at deliberately.

The rule I keep coming back to

A tile must never lie. That sounds obvious, and it is where almost every fault in this booklet ends up. A tile showing the wrong state is worse than a tile showing nothing, because you act on it. My lock reported "locked" for a full morning while the door was open, and my house reported using zero electricity while the kettle was on.

In practice that means: colour tiles from the thing that physically knows the answer, not from a value you stored earlier. Chapter five covers what that looks like.

Surface one: the desk kiosk

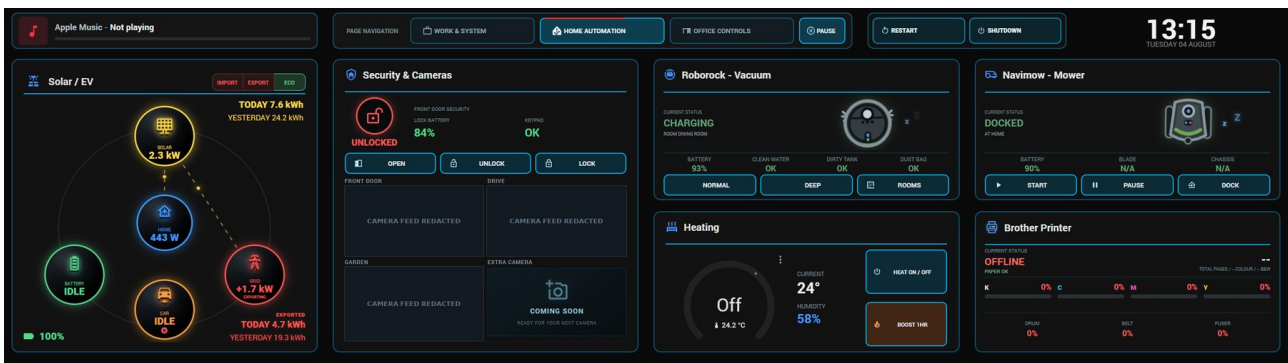
A widescreen panel that cycles through three pages on a one-minute timer: work and system status, home automation, and room controls. It is glanceable furniture rather than something I interact with.

Layout on a very wide screen

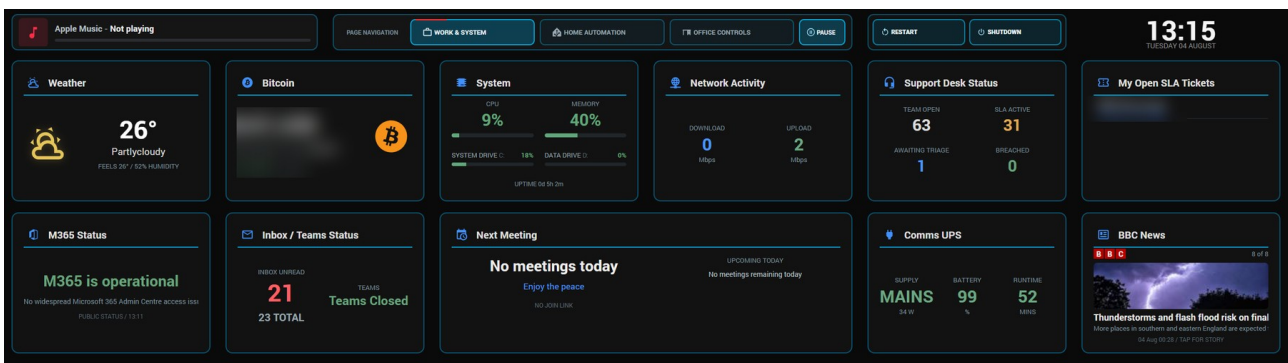
Ordinary card layouts do not suit a 2560-pixel-wide, 720-pixel-tall panel: masonry wants columns, and sections want to stack. What worked was a **single full-screen button-card** with a CSS grid of named areas, each area holding a nested card as a custom field.

```
custom_fields:
  clock: { card: { type: custom:button-card, ... } }
  solar: { card: { type: custom:button-card, ... } }
  network: { card: { type: custom:button-card, ... } }
styles:
  grid:
    - grid-template-areas: "'clock solar network' 'diary solar ups'"
    - grid-template-columns: 1fr 1.4fr 1fr
```

That gives absolute control of a fixed-size display. It has one serious drawback, which is worth stating plainly: **one JavaScript error in any template blanks the entire screen**. Always evaluate a template against live state before saving it (chapter ten shows how). I have blanked that panel more than once, and fixing it blind is no fun.



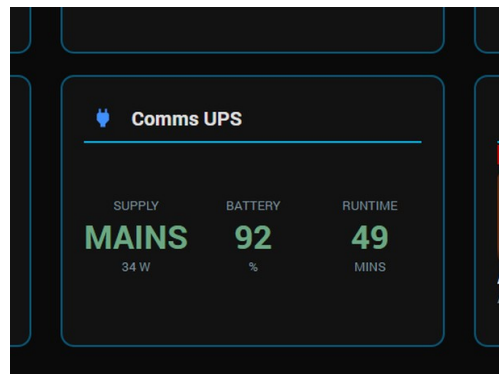
The home automation page in full. Energy flow on the left, then security, the vacuum and mower, heating and the printer. Camera thumbnails are redacted here; on the panel they are live. Note the navigation pills along the top - the active one carries the countdown ring.



The work and system page. Weather, system load, network throughput, support-desk figures, mail and calendar, UPS status and news. Financial values and a ticket reference are redacted. Everything in the top row is a separate nested card in one CSS grid.



The office controls page - desk lighting, a clock plug, air purifier, air conditioning by infrared, and speaker input selection. Two tiles are deliberately reserved for future kit, which keeps the grid balanced.



A widget in the house style: three figures with grey labels, colour-coded by threshold. Green above 80 per cent, amber above 40, red below. The supply figure turns red and the icon changes to a battery the moment mains power goes.

The panel is driven by a small purpose-built Windows application rather than a browser in kiosk mode. It is covered in full in **Appendix A**, and its complete source is included in this pack.

Automatic page rotation

Storage-mode dashboards give you nowhere to put a script, so the only place JavaScript can live is inside a button-card template. My first attempt ran the whole rotation engine as a side effect of a card's name: template, ticking on a one-second update timer and returning "PAUSE" as the button label.

It was clever, and it failed in the worst possible way: on one particular page the card stopped re-rendering, and because the engine only existed *during* a render, rotation simply stopped. Everything still looked fine. The countdown ticked into negative numbers with nothing listening.

THE FIX, AND THE GENERAL LESSON

Install a **single persistent loop** once per page load that reads state directly from the hass object, rather than depending on any card rendering. A render stall can then no longer stop it. More broadly: if a mechanism only runs as a side effect of something else, it will eventually stop running and you will not be told. Full code in **06-page-rotation-controller.js**.

A countdown you can see

Auto-rotation is pleasant until you are halfway through reading something and the page changes. The fix was to make the remaining time visible: the active navigation button's existing blue border fills red, clockwise, completing exactly as the page flips.

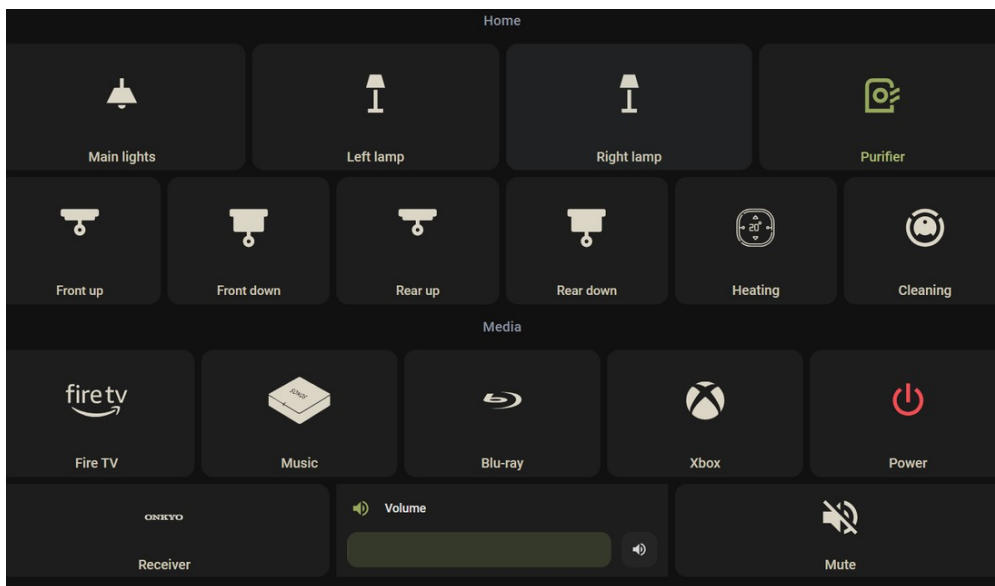
Getting that working taught me two things about animating inside a card, both in chapter ten. The short version: the obvious modern approach - a conic gradient driven by a registered custom property - fails silently inside

shadow DOM, and an SVG stroke animation works perfectly.

Surface two: the wall tablet



The lounge page. Lights and lamps, blind controls, heating and cleaning, then the media row with device logos, and a volume slider along the bottom.



The same page captured from the browser: lights, blinds, heating and cleaning, then the media row and the receiver volume slider. Olive means active - here the purifier is running and everything else is at rest.

Panel, not masonry

My tiles kept coming out at about 117 pixels wide instead of the 150 I had specified, wasting a third of the screen. The cause: masonry views cap column width. On a tablet, use `type: panel` with a vertical-stack inside, and you get the full width.

On a phone, masonry is fine - the screen is narrower than the cap anyway - so the two dashboards differ here deliberately.

Templates, not repetition

Every tile derives from two templates: one for stateful things and one for section headings. Changing a corner radius or a colour then means editing one place rather than forty. Full definitions are in **04-tile-design-system.yaml**.

One subtlety worth knowing if you have a thermostat: a climate entity's **state is its mode**, not whether it is currently calling for heat. Set to "heat", it reads "on" all summer. The attribute you actually want is `hvac_action`, which tells you whether the boiler is running. The template prefers that where it exists.

Activity tiles: the idea I would most like you to steal

Five tiles - streaming, music, disc, console, everything off. Tapping one sets up the whole room. The interesting part is how they know which is active.

The wrong way, which I built first: store the current activity in an `input_select`, set it from each script, colour the tiles from that. It lies. Mine only ever held "Fire TV" or "Music", so when I added a console and a disc player the music tile stayed lit while I was gaming. It also survives you turning things off by hand, so the dashboard cheerfully reports an activity that ended hours ago.

The right way: colour the tiles from the **amplifier's actual source**. It is the one device that cannot be wrong, because it is physically switching the signal. Only one tile can be lit, and it is correct by construction.

```
var e = states['media_player.lounge_receiver'];
var src = (e && e.attributes) ? (e.attributes.source || '') : '';
var on = (e && e.state !== 'off' && e.state !== 'unavailable'
  && /STRM/i.test(src));
return on ? '#94a860' : '#ddd6c8';
```

Amplifier source strings are peculiar - mine include things like `VIDEO3 ... GAME/TV ... GAME`, with three middle-dot characters. Match a distinctive fragment with a regular expression rather than the whole string. Read your own list from Developer Tools first.



The music page, reached from the media row. Album art pulled from the speaker, transport controls, favourites, and an olive volume slider - recolouring which took an embarrassing amount of investigation. See chapter ten.

Mounting, briefly

- A **right-angle magnetic USB-C** lead, routed behind the stand, keeps the cable invisible and lets you lift the tablet off. Check the adapter supports enough power - some are data-only and will slowly lose the battle on an always-on display.

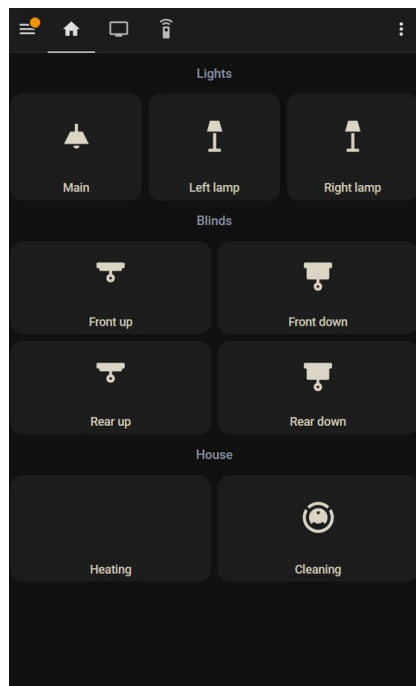
- **Tap to wake** plus a ten-minute auto-lock beats fighting the operating system over brightness. Home Assistant cannot set screen brightness on iOS at all; on Android it can. If you want scheduled dimming on an iPad, use Apple's own Shortcuts app.
- Consider turning the passcode **off** if the tablet lives in a family room and only ever shows the dashboard. Combined with Guided Access it then behaves like a fitted wall panel rather than a tablet.

Surface three: the phone

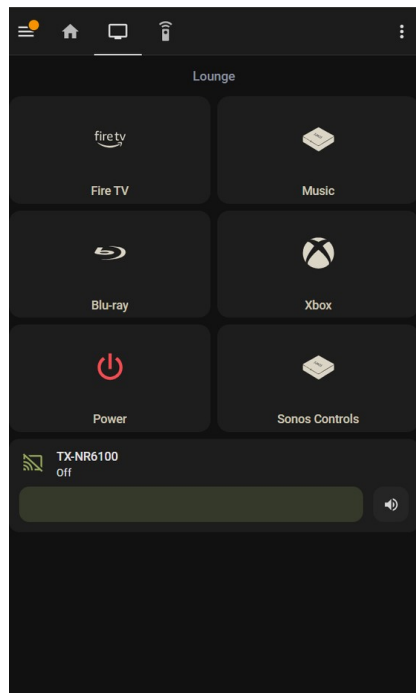
The phone version is not a shrunk tablet. It is a different design brief: it goes everywhere, so it can assume nothing about where you are standing.

What changes

- **Three top-level views only** - Home, Media, Remote. A tab bar with seven entries is unusable on a phone.
- **Detail pages become subviews.** Setting `subview: true` keeps a page out of the tab bar and gives it a proper back arrow. The cleaning room-picker and the music page both live there.
- **No default room.** The tablet can assume the lounge. The phone offers every room equally - the cleaning page lists all of them rather than putting the lounge first.
- **Fewer decorative tiles.** The air purifier lives on eco mode permanently and never needed a tile; it came off the phone entirely.



The phone home view. Same palette and icons, smaller tiles. Lights three across, blinds two-by-two, then heating and cleaning. The tab bar at the top carries only three entries.



The media view. The four activity tiles, Power, and a Sonos Controls tile that opens the music subview - a visible tile rather than a hold action, for the reason at the end of this chapter. The amplifier volume slider sits below.

An audio zone picker

With speakers in two rooms you need three behaviours: play here, play there, or play in both. Putting a favourites row under each speaker duplicates every tile and still does not give you "both".

The answer was to copy the manufacturer's own app: **tick the rooms, then pick what to play**. Two helpers hold the selection; one script does the work, grouping the speakers automatically when both are ticked and ungrouping when only one is. Code in **02-battery-and-audio-scripts.yaml**.

A USER-EXPERIENCE LESSON I WALKED STRAIGHT INTO

I built the zone ticks, tested that the helpers toggled, and declared it working. Then I tapped a favourite and nothing happened - because no room was ticked, and the script correctly did nothing. **A mode selector that produces no visible change feels broken**. The fix was a templated heading that reads "Favourites - both rooms" or "Favourites - tick a room above first". Hidden state must be visible somewhere.

Two smaller touches that earn their keep

- **A loading spinner** on each activity tile while its script runs. Without it people press twice. A sixteen-pixel CSS ring, shown only when the script state is on.
- **A now-playing strip** that appears on the Media page only when music is the active source, using a conditional card. Album art, play/pause and next - so the common case needs no navigation at all, and the page stays short otherwise.

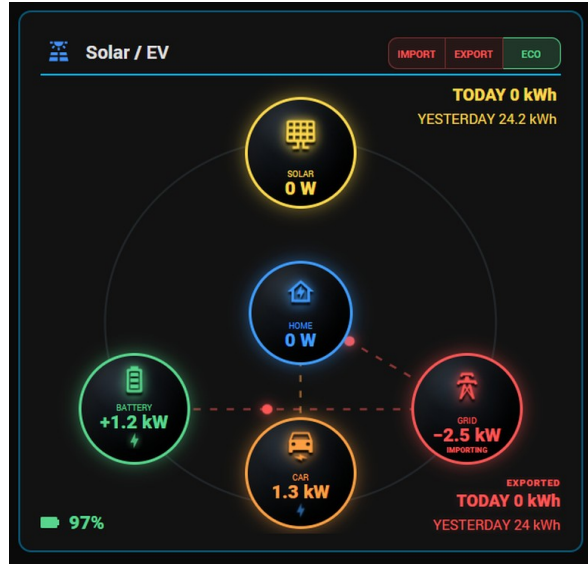
And one thing I got wrong

I put "open the music page" on the **hold** action of the Music tile, copying the tablet. A deliberate press tips over Home Assistant's roughly 500-millisecond hold threshold, so tapping Music navigated instead of playing music. I caught myself out with my own dashboard, twice, before working out why.

If an action matters, give it a visible tile. Hold actions are for extras, not for anything you need reliably.

The energy flow card

Five circles on a ring - solar, house, battery, grid, car - with animated lines showing where power is flowing. It is the card I look at most, and the one that has caused the most trouble, because it does **arithmetic on sensor readings**.



Reading correctly at midnight during the cheap-rate window: no solar, importing from the grid, battery charging, car idle but plugged in. Getting that last figure to tell the truth took three attempts.

Design details worth copying

- **Thresholds, not gradients.** Battery green above 80 per cent, amber 20 to 80, red below. Faster to read than a continuous scale.
- **A deadband on discharge.** A battery reports 15 to 20 watts of self-consumption constantly; without a 300-watt floor the discharge animation runs all night. The deadband relaxes to 20 watts during a deliberate forced export, when you do want to see small flows.
- **Hide the trickle at 100 per cent.** A full battery accepting 60 watts of surplus solar is noise. Above 99 per cent the node reads IDLE and no flow line is drawn.

The trap: subtracting the car from the house

The card worked out the house figure as $\text{load} - \text{car}$, on the reasoning that the inverter's load reading includes everything in the house including the charger. Sound logic. It failed three times.

Once when the charger stalled and reported a phantom 7.2kW session into a car that was already full, with its internal clock seven hours adrift. The subtraction produced a negative number, clamped to zero, and my house appeared to use no electricity.

Once when I briefly had two Home Assistant instances polling the same inverter during a migration. Modbus does not tolerate that; values land in the wrong registers.

And once for the real reason, which took longest to see: my charger reports `charging_state: "Charging"` whenever **a cable is attached**, whether or not any current flows. The card trusted that word, inferred a session, derived a plausible figure, and subtracted it. It only misbehaved during the overnight battery charge, because that is when there was a large grid import to misattribute - and I was asleep for it every night for weeks.

THE FIX, IN ONE LINE

Trust **measured power**, never a state word. Zero watts means nothing is charging - no inference, no estimation, nothing to go stale. The same principle fixed the car icon: pulsing bolt when power is flowing, static plug when merely connected, red cross when unplugged. Three states, none of them guessed.

A test worth running on your own sensors

Before building anything that does arithmetic, check that your understanding of your own sensors is correct. Energy must balance:

```
solar + grid_import + battery_discharge  
  approx equals  
house_load + battery_charge + grid_export
```

If that does not add up to within a few per cent, something is misread - a sign convention, a duplicated figure, a sensor that means something other than you think. Far better to find out now than at two in the morning.

Fuller notes, including the icon logic and a plausibility check you can apply to any derived figure, are in **07-energy-flow-card.md**.

Scripts that feel instant

Pressing "watch television" has to wake the TV, power the amplifier, switch its input, wake the streaming box and stop whatever was playing. Written in the obvious order, that took between twenty and forty seconds, because you end up waiting for the television before you even start on the amplifier.

Parallel branches

Restructured as parallel branches - one per device - the whole thing takes as long as the slowest single device, which is usually the TV's own wake time. Same result, a third of the wait, and it feels like a different system.

```
mode: restart
sequence:
  - parallel:
    - sequence:      # television: wake, wait, select input
    - sequence:      # amplifier: power, wait, select source
    - sequence:      # source device: wake, send HOME
    - sequence:      # pause music if it is playing
```

mode: restart	Tapping twice restarts cleanly instead of queueing two runs.
continue_on_error	One unreachable device must not abandon the whole script.
wait_template	Wait for the device to be <i>ready</i> rather than guessing a delay. With <code>continue_on_timeout</code> so a slow TV cannot block the rest.
Guard conditions	Only wake the console if it is actually off. Tapping the tile mid-game should do nothing.

A note on waking things

Wake-on-LAN works for televisions. It does **not** wake an Xbox - that uses Microsoft's own protocol, and I wasted a while on magic packets before discovering it. The official integration exposes a remote entity with a `WakeUp` command, which works reliably. The console must be in sleep mode rather than energy-saving, and expect roughly a minute's lag before the state updates, because it reports via the vendor's cloud.

For devices with no network control at all - my disc player - a smart infrared blaster gives you power control for very little money. It is one-way, so the entity state reads "unknown" forever. You can send but never confirm.

The discipline of turning off

My "everything off" script turned off the television, the streaming box and the amplifier. I was pleased with it. It did not pause the **speaker**.

One night I pressed it at 01:33 with music playing. The music kept streaming silently, because the amplifier was off. At 02:59 the amplifier powered itself on - an unrelated quirk of that unit - and the house filled with Bruce Springsteen. I woke at 04:44 wondering what on earth was happening.

THE RULE THAT CAME OUT OF IT

Pause audio first, before anything else, and pause it from every activity script rather than only the off button. If something can play silently, one day it will play loudly at the worst possible moment. There is also a belt-and-braces automation in **03-automations-and-watchdogs.yaml** that stops overnight playback when the amplifier is off.

Order matters more than you would think

The music script turns the television off, because music through a speaker needs no screen. That failed intermittently until I understood why: if the games console was still awake and pushing HDMI, the TV either ignored the standby command or woke straight back up.

The working order is: sleep the console, **wait two seconds**, then turn off the television. Let the HDMI signal go quiet before asking the display to sleep.

Watchdogs

Most automation is about making things happen. The automations I value most do the opposite: they notice when something has **stopped** happening. That is much harder, because a broken integration looks identical to a quiet house.

The signature to look for

Every one of these faults had the same shape, and once you can spot it you will find it everywhere:

ALIVE BUT SILENT

Polled data keeps updating while pushed data stops. The integration reports as loaded, the device works in its own app, nothing appears in any log - and yet the events you care about never arrive. Look for one sensor moving while a related one is frozen. That mismatch is the tell.

Three real examples

Doorbell	The cloud push listener died. Its activity sensor kept updating while the ding and motion events sat frozen from the previous day, so the doorbell simply never rang in Home Assistant. Nothing in any log. The watchdog fires only when activity is <i>recent</i> but events are <i>stale</i> - that double condition is what stops it firing during a quiet week away.
EV charger	Its local API stops responding roughly once a day while the cloud connection stays up, so the phone app looks perfectly fine. Detected the same way: charger sensors frozen while the inverter keeps reporting.
UPS	Moving the unit disturbed the USB cable. The integration served its last known values indefinitely - zero watts, a 78-minute runtime - which looked like real data. The give-away was that every reading shared an identical timestamp an hour old.

Which leads to the most useful habit in this booklet

Check timestamps, not values. When something looks wrong, the first question is not "what does this sensor say" but "when did it last change". Half the faults here announced themselves the moment I looked at `last_changed`.

In Developer Tools, or from a template:

```
{{ states.sensor.my_sensor.last_changed }}
{{ (as_timestamp(now())
  - as_timestamp(states.sensor.my_sensor.last_changed, 0)) / 60 }} minutes ago
```

A caution about reloading things automatically

A watchdog that reloads an integration is a blunt instrument. Give it a wide margin - mine waits twelve hours - and have it notify you when it acts, so a recurring fault does not stay invisible behind a self-healing loop. A watchdog that silently papers over a daily failure is worse than no watchdog, because you never find the cause.

Styling inside shadow DOM

Two problems took longer than anything else in this project, and both come down to the same cause: Home Assistant's cards live inside **shadow DOM**, and some CSS features do not cross that boundary. Both fail *silently*, which is what makes them expensive.

Problem one: recolouring a native slider

I wanted the volume slider on a tile card to be olive instead of Home Assistant's blue. The obvious approach did nothing:

```
card_mod:
  style: |
    ha-card { --control-slider-color: #94a860 !important; }
```

Tracing the variable up the element tree showed it inheriting correctly all the way down - and then the slider element itself overriding it, because it derives its colour from `--tile-color`, which Home Assistant sets **inline on the card** from the entity's *state* colour. That also explained why it looked blue when the amplifier was on and grey when it was off: I was not fighting a theme, I was fighting the entity state.

```
card_mod:
  style: |
    ha-card {
      --tile-color: #94a860 !important;      /* the one that actually matters */
      --feature-color: #94a860 !important;
      --control-slider-color: #94a860 !important;
      --control-slider-background: #ddd6c8 !important;
      --control-slider-background-opacity: 0.18 !important;
    }
```

Style at the `ha-card` level only. My attempts to reach deeper collapsed the card and displaced its neighbours. Twice.

Problem two: animating a progress ring

For the rotation countdown I reached for the standard modern technique: a conic gradient masked to the border, animated through a registered custom property. The pseudo-element appeared; the gradient and animation did nothing at all.

WHY IT FAILED

`@property` is **document-scoped**. Registering it inside a card's shadow root is silently ignored, so the custom property was invalid, which made the whole conic gradient invalid, which killed the background - and with nothing to animate, the animation never ran either. No error, no warning, nothing in the console.

The answer is **SVG stroke animation**, which needs no custom properties and works happily inside shadow DOM:

```
'<rect x="1" y="1" rx="9" fill="none" stroke="#ff2f2f" stroke-width="3"'
+ ' pathLength="100" stroke-dasharray="100" stroke-dashoffset="100"'
+ ' style="animation:dmsw ' + total + 'ms linear forwards;'
+ ' animation-delay:-' + elapsed + 'ms"/>'
```

Three details in there, each of which mattered:

- **pathLength="100"** normalises the path, so the dash values work at any element size. No perimeter arithmetic.
- **Position the overlay at inset -2px.** An absolutely positioned child sits inside the *padding* box, so inset 0 draws the line inboard of the border - it reads as a second line rather than the border filling up.
- **A negative animation-delay.** The card re-renders periodically, and every re-render restarts a CSS animation from zero. The symptom was a line that crept forward, snapped back, crept slightly further, snapped back. Setting a fixed duration with a negative delay equal to the elapsed time makes every re-render land exactly where it should.

Always pre-flight a template

One error in one template can blank an entire view. Evaluate against live state *before* writing the config:

```
const inner = template.replace(/^s*\[\\[\\/, ''\].replace(/\\]\\\]s*$/, '');
const result = new Function('states', 'entity', 'user', 'hass', inner)(
  hass.states, null, hass.user, hass);
// only save if result is a non-empty string
```

This caught a real error for me - a variable I was reassigning turned out to sit inside a const declaration list. Had I saved it, the entire energy card would have gone blank.

Fuller write-up, with the diagnostic output that led to each conclusion, in **05-styling-inside-shadow-dom.md**.

Moving to new hardware

The server outgrew its original home - a thin client down to 3.4GB of free space, with backups being deleted to make room. Moving to a small refurbished desktop took an evening and went mostly smoothly. The parts that did not are the useful bit.

The procedure

- | | |
|-----------------------|---|
| 1. Full backup | Emphasis on full . Mine turned out to be a <i>partial</i> and I had deleted the older ones for space - which would have left me restoring without add-ons. Check the type before you rely on it. |
| 2. Match install type | Home Assistant OS to Home Assistant OS. Same architecture means a like-for-like restore with nothing to convert. |
| 3. Write the image | Boot a Linux live USB on the new machine and write the OS image to its internal disk. Check device names twice - the wrong one destroys whatever is on it. Secure Boot off, UEFI on. |
| 4. Restore | At the onboarding screen choose restore from backup rather than creating an account. Everything comes back: add-ons, HACS, dashboards, history. |
| 5. Take the address | Only after the old machine is off. Everything points at that IP. |

DO NOT LET BOTH RUN FOR LONG

While both instances are live they will both poll your devices. My inverter is read over Modbus, which does not tolerate two pollers - values landed in the wrong registers and my energy card produced nonsense. Verify the new machine, then shut the old one down before doing anything else.

The four things that broke

All four looked healthy from the outside, which is the theme of this entire booklet.

- | | |
|--------------------|--|
| MQTT lock | Reported "locked" all morning while the door was open. Two causes stacked: the broker add-on's login did not survive the restore , so the lock could not authenticate - and the restore had brought over 1,013 retained MQTT messages , so Home Assistant was displaying a ghost of the old state. Nothing appeared in the broker log at all, not even a rejected connection, which sent me hunting for network problems that did not exist. |
| UPS | The NUT integration reported DATA-STALE. Reloading the integration did nothing: the add-on needed restarting, because its USB driver had lost the device when I moved the unit and does not re-enumerate on its own. |
| Speakers | Reported state correctly but would not accept commands, because the integration held stale connections from before the move. A reload fixed it in seconds, once I stopped assuming the fault was elsewhere. |
| Cloud integrations | The lighting integration needed re-authenticating, and the doorbell asked for confirmation - both normal when a token moves to new hardware. Not faults, but worth expecting. |

A checklist for afterwards

- **Add-on logins and credentials** - check them explicitly rather than assuming they came across.
- **Retained MQTT messages** - anything MQTT-based may show convincing stale data until the device reconnects and publishes afresh.
- **USB devices** - restart the owning add-on, not just the integration.
- **Anything that registered a callback with your IP** - doorbells, locks, bridges. If you changed address, they are still talking to the old one.
- **Your phone app** - mine still pointed at the temporary address for hours, showing cached values that contradicted the server and confused two separate diagnoses.

And one final trap, which was entirely my own

Midway through, my diagnostic tools began reporting hour-old values and hanging on every request. I concluded the new server was struggling, and spent a good while investigating a machine that was perfectly healthy.

The actual cause was a **stale browser session**. A fresh tab answered in eight milliseconds. If your tooling starts insisting that working things are broken, suspect the tooling.

Everything I would tell myself

The collected list, in rough order of how much time each would have saved me.

On trusting your data

- **Check timestamps before values.** "When did this last change" finds more faults than "what does this say".
- **Stale data looks exactly like real data.** Retained MQTT messages, frozen integrations and cached app views all present confident, plausible, wrong numbers.
- **A state word is not a measurement.** "Charging" meant "cable attached" on my hardware. Where a number exists, trust the number.
- **Derive from the thing that physically knows.** The amplifier cannot be wrong about its own input. A helper you set earlier can be wrong about everything.
- **Make the energy balance.** If inputs do not equal outputs, you have misunderstood a sensor.

On writing frontend code

- **Pre-flight every template.** One error can blank a whole view, and a full-screen kiosk card means fixing it blind.
- **@property does not work inside shadow DOM.** It is document-scoped and fails silently. Use SVG stroke animation instead.
- **Sliders take their colour from --tile-color,** set inline from entity state. Override that, with !important.
- **Re-renders restart CSS animations.** Use a fixed duration and a negative delay so any re-render resumes in the right place.
- **Keyframes must travel with the element that uses them.** Define them unconditionally, or they vanish when the branch that contained them does.
- **Do not hide important actions behind hold.** A deliberate press crosses the threshold and you get the wrong action.

On making things reliable

- **Never rely on a mechanism that only runs as a side effect.** Install a persistent loop that reads state directly.
- **Pause audio first.** Anything that can play silently will eventually play loudly at 3am.
- **Guard every wake and sleep.** Only wake what is off; only sleep what is on.
- **Prefer local control.** Faster, and it survives the vendor going out of business - which mine did.
- **Do not update firmware you cannot replace.**
- **Put the router and the modem on the UPS,** not just the server. Otherwise your power-cut notification has nowhere to go.

On design

- **Two colours are enough.** Eight is a fruit machine, and slower to read.
- **Hidden state must be visible.** A mode selector with no visible effect will be reported as broken - correctly.
- **Different screens need different briefs.** A phone is not a small tablet; it cannot assume where you are.
- **Show progress.** A spinner while a script runs stops people pressing twice.
- **Let the numbers be specific.** "LOW 19GB" is more useful than "LOW".

On the process itself

- **Use the logbook.** It annotates state changes with what caused them, which settled the 3am music mystery in one query - the amplifier had switched itself on with no involvement from Home Assistant at all.
- **Write things down as you go.** The hour I lost to the lock was mostly not knowing how it had originally been configured.
- **Live with it before extending it.** Every change in this booklet that I am pleased with came from an actual irritation. The ones I invented at a keyboard mostly got reverted.

The code

Everything referenced here is in the accompanying **code** folder, genericised and commented. Entity IDs are placeholders - read your own from Developer Tools and substitute.

01-activity-scripts.yaml	The parallel-branch AV activity pattern: streaming, music, console, disc and everything-off, with the guards, waits and the audio-pause discipline.
02-battery-and-audio-scripts.yaml	Battery mode toggles for a solar system, and the multi-room audio zone picker with automatic grouping.
03-automations-and-watchdogs.yaml	UPS power-cut alerts, the doorbell event watchdog, an EV charger stall detector, a low-storage warning and the overnight silent-audio guard.
04-tile-design-system.yaml	The button-card templates, the activity-tile logic, dynamic headings, warning badges, and the layout notes about panel versus masonry.
05-styling-inside-shadow-dom.md	The slider colour fix and the progress-ring technique, with the diagnostic output that led to each conclusion.
06-page-rotation-controller.js	The persistent rotation controller and the countdown ring, with notes on using it as a watchdog over an existing engine.
07-energy-flow-card.md	Flow card structure, thresholds and deadbands, the EV subtraction trap and a plausibility check for derived figures.
08-kiosk-host-app.md	The Windows kiosk host: why build one, how it is structured, and simpler alternatives if you would rather not.

If you use any of it

Take it, change it, publish your own version. It is all here because other people's write-ups saved me time, and this is the return leg.

The one thing I would ask: if you find that one of these patterns fails on your hardware, say so publicly. Half the value of this booklet is the faults, and the only reason I could document them is that they happened in a real house with real kit that did not behave as documented.

Built and broken and rebuilt over several months. Every fault described here is one I actually had.

APPENDIX A

The kiosk host application

The widescreen panel is driven by a small purpose-built Windows application. **The full source is included in this pack**, under app/XEON-Dashboardv3/. It is MIT-spirited: take it, rename it, do as you like.

THE SAME DISCLAIMER APPLIES, DOUBLY

This application was written almost entirely by AI, working to my specification. It runs here 24 hours a day and has done for months, but it has not been reviewed by anyone else, has no test suite, and is not code-signed - so Windows SmartScreen will warn about it. **Read it before you run it**, and build it yourself rather than trusting a binary from a stranger.

Why build one at all?

It began as PowerShell launching Edge in kiosk mode, which worked until it did not: an update banner, a crash-restore bar, a lost focus, a stray keypress opening a tab. The reasons to write a host:

- **It survives.** No browser chrome, no update prompts, no way to navigate away.
- **It knows which monitor it belongs on.** Display indices shuffle when you plug something in; it matches on the monitor's *name* and reasserts itself if the layout changes.
- **It starts with Windows and hides.** No console window, a tray icon for the rare occasion you need it.
- **It reconnects.** If Home Assistant restarts or the network blips, it retries rather than sitting on an error page.

The stack, and why

WPF on .NET 8	Chosen over WinUI 3 deliberately. For a 24/7 borderless single-monitor kiosk that must publish as a self-contained executable needing no admin rights, WPF meets every requirement with less friction. MVVM, dependency injection and Serilog throughout.
WebView2	The Chromium engine already present on Windows 10 (2004+) and 11. The app detects it and can provision it if missing.
Self-contained publish	A single executable of roughly 70MB with the runtime bundled, so nobody has to install .NET. An Inno Setup script wraps it with Start Menu and startup shortcuts, no admin required.

How the source is arranged

app/XEON-DashboardV3/ README.md build.ps1 XEON-Dashboard.sln Docs/ ARCHITECTURE.md PACKAGING.md ROADMAP.md settings.sample.json Installer/ XeonDashboard.iss src/XeonDashboard/ App.xaml(.cs) Views/ DashboardWindow.xaml SettingsWindow.xaml ViewModels/ Services/ Models/ Helpers/	build and configuration notes one command: portable exe + installer framework choice, design decisions publishing, SmartScreen, code signing copy to settings.json and edit Inno Setup script host builder, DI, Serilog, single-instance lock borderless WebView2 host first-run setup, live monitor picker navigation policy, reconnect logic monitor, tray, settings, startup, runtime settings model, Win32 interop
---	---

Every service sits behind an interface, which keeps the WPF layer thin. If you would rather use WinUI or Avalonia, the service layer is framework-agnostic and can be lifted as it stands.

Configuration

A single settings.json, editable from the settings window or by hand:

```
{
  "DashboardUrl": "http://homeassistant.local:8123/lovelace/0?kiosk",
  "MonitorName": "XENEON EDGE",
  "LaunchDelaySeconds": 10,
  "ReconnectIntervalSeconds": 5,
  "HideMouse": false,
  "AlwaysOnTop": false,
  "StartWithWindows": false,
  "ExitSilentlyIfMonitorMissing": false
}
```

Set `MonitorName` to an empty string to fall back to the primary display - otherwise it looks for a monitor literally named XENEON EDGE and you will wonder why nothing appears.

The decisions that mattered

- **Native resolution, no scaling.** The app is a browser viewport; it cannot re-flow your dashboard. If the layout looks wrong, fix it in Home Assistant. Trying to solve layout in the host is a trap, and the settings screen says so out loud.
- **Find the monitor by name, not index.** With a fallback, or nobody else can run your build.
- **Single-instance lock.** Two copies fighting over one fullscreen window is a miserable bug.
- **Keep the tray icon.** The temptation is to make an appliance completely inescapable. Resist it - you will need to reach settings, and doing that over remote desktop with no visible UI is horrible.
- **The host stays dumb.** It navigates once and stays there. All the page cycling happens inside Home Assistant, so the dashboard behaves identically in any browser and the host has nothing to go wrong.

Things worth knowing before you run it

- **SmartScreen will warn** about an unsigned executable. Build from source and you know what you are running.
- **The kiosk is a separate browser session** from your desktop. When the panel looks stuck but your browser is fine, reload the panel. I lost an hour to this.
- **Signed camera URLs expire.** A page held open for weeks will retry with stale tokens after a server restart, logging authentication warnings that look alarming and are harmless.
- **Long-lived pages accumulate problems.** A nightly refresh costs nothing and prevents a whole category of mystery.
- **The publish folder holds a browser profile** containing your Home Assistant session. Never share it - only source. That is why this pack excludes it.

If you would rather not run a Windows app

Entirely reasonable. A **Raspberry Pi with Chromium in kiosk mode** and a systemd unit is the cheapest well-trodden route. An **old tablet** running the companion app takes no effort at all and gives you touch. **Fully Kiosk Browser** on Android is genuinely excellent, with motion wake and remote admin built in.

I wrote a host because the panel is a Windows machine that also does other things, and I wanted it to behave like fitted furniture. If your display is dedicated hardware, one of the above will get you there faster.